

Tour Ceylon

Tourist SOS Dashboard Guide

A simple explanation of the emergency app — features, files, tech, and database.

Read this first

Tourist SOS is the emergency phone-style app. The visitor does not create a new login. They sign in on Travel, tap SOS, and this app opens already logged in. Tapping an SOS type sends their GPS to the nearest police station.

1. What is the Tourist SOS dashboard?

It is a mobile-first emergency app for visitors in Sri Lanka. The visitor taps one of six SOS types (Emergency, Medical, Assault, Theft, Accident, Harassment). The system finds the nearest police station, creates a case, lets the visitor track status, chat with that station, and optionally call the nearest hospital.

2. Why so many files?

Same idea as Travel: small files with one job each. Pages are screens. Components are reusable pieces (SOS bar, chat, workflow). API helpers talk to the server. Backend routes handle create-incident, chat, profile. Models map to SOS database tables.

- pages/ — Home, History, Profile, Enter (handoff landing)
- components/ — SOS buttons, chat, workflow steps, hospital call, nav
- api/ — tourist token + handoff from Travel
- hooks/ — get GPS once (fresh location)
- backend sos_models + routes — incidents, chat, hospitals, tourists

3. Tech stack — what we use and why

React 18	Frontend	Interactive mobile screens
Vite	Frontend tool	Fast local server (port 5175)
React Router	Frontend	Home / History / Profile routes
Axios	Frontend	API calls with tourist JWT
Lucide icons	Frontend	Clear emergency icons
vite-plugin-pwa	Frontend	Install app on phone home screen
Flask + JWT	Backend	Secure tourist APIs
SQLAlchemy sos_models	Backend	SOS tables in Postgres
PostgreSQL	Database	Incidents, chat, profile data

4. How login works (no separate SOS password)

- User logs into Travel (Tour Ceylon).
- Taps SOS in the Travel header.
- Travel calls bridge-travel → server finds/creates tourist by email → tourist JWT.
- Travel opens SOS at /enter?handoff=TOKEN.
- SOS saves touristToken and goes to Home.
- If token is missing, SOS redirects back to Travel login.

5. How an SOS reaches the police (database path)

Visitor taps SOS → browser gets fresh GPS → POST /api/incidents → Flask finds nearest station → creates incidents row (status open) + one location_ping → notifies station / emergency contact / hotel → Police dashboard for that station shows the case.

*Status steps: **open** → **acknowledged** → **dispatched** → **closed**. Officers change status in the Police app. Tourist only watches and chats.*

6. Features explained clearly

6.1 Enter / handoff

Landing page that reads the token from Travel and opens Home. Login/Register pages only redirect to Travel.

6.2 Home + SOS trigger

Main screen with radar look. Bottom bar has 6 SOS types. Tap one → GPS → case created → confirmation overlay.

6.3 Workflow

Shows police progress: SOS sent → Acknowledged → Dispatched → Closed. Polls the server every few seconds.

6.4 Chat

After SOS, tourist can send text, photo, or voice to the assigned station only.

6.5 History

Lists current open case, other active cases, and closed cases.

6.6 Profile

Edit name, phone, passport/NIC, hotel, emergency contact, photos — used when police view the case.

6.7 Call nearest hospital

Separate helper: uses GPS to find nearest hospital and opens the phone dialer. Not part of the police case.

7. Frontend files — what each one does

Folder: *frontend/tourist/src/*

main.jsx	Starts app, applies Travel handoff, registers PWA	Entry point
App.jsx	Routes, session check, global SOS trigger	App brain
theme.js	Colors and status styles	Consistent look
api/client.js	Axios + touristToken; 401 → Travel login	API access
api/sessionHandoff.js	Reads URL tokens; redirects to Travel	Cross-app login
api/travelBridge.js	Older travel helpers	Legacy (mostly unused)
pages/Enter.jsx	Receives handoff from Travel → Home	Login bridge page
pages/Login.jsx	Redirects to Travel login	No separate SOS login
pages/Register.jsx	Redirects to Travel register	Same
pages/Home.jsx	Workflow, chat, hospital, recent cases	Main emergency screen
pages/History.jsx	Lists open and closed incidents	Case history
pages/Profile.jsx	Edit visitor + emergency contact	Safety profile
components/SOSButton.jsx	Fixed bar with 6 SOS types	One-tap trigger
components/SOSSentOverlay.jsx	Shows 'SOS sent' confirmation	Clear feedback
components/IncidentWorkflow.jsx	Status stepper UI	Show police progress
components/IncidentChat.jsx	Chat with station	Two-way messages
components/CallNearestHospital.jsx	Dial nearest hospital	Medical help
components/TouristNav.jsx	Tabs: SOS / History / Profile + Trips	Navigation
components/AmbientBackground.jsx	Background / radar visuals	Atmosphere
hooks/useGeolocation.js	Gets GPS once (no tracking loop)	Fresh location for SOS

8. Backend files — what each one does

sos_models.py	All SOS database models/tables	One place for SOS schema
routes/tourists.py	Bridge from Travel, profile, my incidents	Tourist identity
routes/incidents.py	Create SOS + status + officer transitions	Case lifecycle
routes/chat.py	Send/receive messages	Chat API
routes/hospitals.py	Nearest hospital lookup	Hospital dial
routes/locations.py	Location pings (continuous ping disabled)	GPS storage
routes/notifications.py	Officer re-notify contact/hotel	Extra alerts
services/notify.py	Send alerts when SOS is created	Notify station/contact
services/routing.py	Nearest station / hospital math	Geo matching
seed_sos_data.py	Seed stations, hospitals, officers	Demo / setup data

9. Database tables

tourists	Visitor account and profile
emergency_contacts	Person to notify
incidents	SOS cases and status
stations	Police stations
location_pings	GPS captured at SOS time
chat_messages	Tourist ↔ station messages
hospitals	Hospitals for nearest dial
hotels	Optional hotel notify
officers	Used by Police app (same database)

10. Where data lives

localStorage	touristToken, touristId, activeIncidentId (UI cache)
PostgreSQL	Real incident status, chat, profile — source of truth

Remember

Travel hands off a tourist session → visitor taps SOS with GPS → nearest station gets the incident in the database → tourist watches status and chats. Hospital dial is separate.